ARGUS

Autonomous Threat Detection and Response for IoT Networks

IoT Project Report

Submitted in Partial Fulfilment of the requirements
for
IoT laboratory course
of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE & ENGINEERING

by

PREET MAKADIYA
(23BCP414)

OM KATHIRIYA
(23BCP417)

VANSH PAUN
(23BCP413)

Under the Guidance of

DR. AMIT SINGH
DR. RAJENDRA CHOUDHARY



Department of Computer Science & Engineering,
School of Technology,
Pandit Deendayal Energy University, Gandhinagar 382 426

Nov 2026

Certificate of Originality of Work

We hereby declare that the B. Tech. IoT Project entitled “ARGUS – Autonomous Threat Detection and Response for IoT Networks” submitted by us for the partial fulfilment of the IoT laboratory course of Bachelor of Technology to the Dept. of Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by us under the supervision of Dr. Amit Singh and Dr. Rajendra Choudhary.

We also declare that this written submission adheres to the university guidelines for originality and includes proper citations and references wherever required.

We also declare that we have maintained high academic honesty and integrity and have not falsified any data in our submission.

We also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Signature of the Students	**Signature of the Supervisors**
Preet Makadiya Roll Number: 23BCP414	Dr. Amit Singh Faculty Supervisor
Om Kathiriya Roll Number: 23BCP417	Dr. Rajendra Choudhary Faculty Supervisor
Vansh Paun Roll Number: 23BCP413	

Place: Gandhinagar

Date:

Acknowledgement

This project is the result of teamwork, guidance and support from many people, and we are happy to thank them here.

We express our sincere gratitude to our supervisors, **Dr. Amit Singh** and **Dr. Rajendra Choudhary**, for their guidance, useful suggestions and constant encouragement at every stage of this work. Their questions pushed us to make ARGUS simpler to explain and stronger in what it does.

We thank the Department of Computer Science & Engineering, School of Technology, Pandit Deendayal Energy University, for providing the laboratory facilities and a supportive environment to carry out this project. We also thank the laboratory staff for their help during hardware bring-up and testing.

We are grateful to our classmates and friends for their feedback during demonstrations, and to our families for their patience and encouragement.

Preet Makadiya (23BCP414)
Om Kathiriya (23BCP417)
Vansh Paun (23BCP413)

INDEX

Topic No.	Title	Page No.
	Certificate of Originality of Work	i
	Acknowledgement	ii
	Index	iii
1	**Introduction**	**1**
1.1	Introduction / Background	1
1.2	Motivation	1
1.3	Objective	1
1.4	Problem Statement and Proposed Solution	2
1.5	Approach	2
1.6	Scope of the Project	3
1.7	Organization of the Rest of the Report	3
2	**Literature Review**	**4**
2.1	Previous Approaches to Solve the Problem	4
3	**Hardware Design**	**6**
3.1	Overview of the Two-Node Setup	6
3.2	Architecture of the Defending Model	6
3.3	Defending Node: Components and Wiring	8
3.4	Attacker Node	11
3.5	Test Network	11
4	**Software Design**	**12**
4.1	Software Environment	12
4.2	Detection Layer: The 13 Modules	12
4.3	Zero-Trust and Replay Protection	13
4.4	Correlation Engine	13
4.5	Response Layer	14
4.6	Tamper-Evident Forensic Log	14
4.7	Dashboard	14
4.8	Attacker Simulator Software	17
5	**Results and Discussion**	**19**
5.1	Test Setup	19
5.2	Test Cases and Results	19
5.3	Discussion and Limitations	20
6	**Conclusions and Future Scope**	**21**
6.1	Conclusions	21
6.2	Future Scope	21
	References	**22**
	Appendix	**23**

Chapter 1
Introduction

1.1 Introduction / Background

The Internet of Things (IoT) connects small, cheap devices such as sensors, cameras and smart switches to a network. These devices are everywhere, but they are usually built to be cheap and quick, not safe. Many have weak passwords, no updates and no way to tell the owner that something is wrong. The Mirai botnet showed how serious this can be: it took over large numbers of such devices and used them to launch huge attacks [1], [9].

Big companies protect their networks with dedicated security tools and security teams. A student lab, a small office or a home does not have these. The device is simply left alone, and an attacker can try many tricks: a fake Wi-Fi network, a fake router, wrong data sent to the sensor, or even opening the box and touching the hardware.

ARGUS is our answer to this gap. It is a small "guard" device built on one ESP32 board. It sits next to the IoT device, watches the network and the device itself, decides on its own whether something is a real attack, and protects the device automatically. A second ESP32 board, the **Attacker simulator**, is used only to create safe test attacks so that the guard can be demonstrated and tested.

1.2 Motivation

- **Attacks are fast and mixed.** A real intruder rarely does one thing. In the same minute they may copy the Wi-Fi name, abuse the message broker and send false sensor values.
- **Small setups have no protection.** Professional intrusion detection tools need a computer or a cloud account and an expert to read the alerts.
- **Alerts alone are not enough.** Most student projects detect one symptom and send a message. Nobody may be there to read it in time.
- **Evidence is easy to erase.** After an attack, ordinary log files can be edited, so the owner cannot prove what happened.
- **ESP32 is cheap and capable.** One ESP32 has Wi-Fi, enough memory and enough speed to run many checks and a web page at the same time.

1.3 Objective

The aim of this project is to build a low-cost, self-contained security system for IoT networks. In detail, ARGUS should:

- detect attacks on the **network** (fake Wi-Fi, fake router, link jamming), the **protocol** (MQTT abuse, replayed messages), the **device** (changed firmware, crashes) and the **data** (false sensor values);
- combine the signals from different checks, so that one odd event only gives a **warning** and several related events are treated as a real **attack**;
- **respond automatically** by cutting power to the protected device and alerting the operator;
- keep a **tamper-evident log** that proves what happened;
- **recover by itself** after the danger has passed;

- show everything on a simple, easy-to-read **dashboard** without needing the internet;
- be tested with a second ESP32 that safely simulates the attacks.

1.4 Problem Statement and Proposed Solution

Problem Statement

IoT devices are easy to attack and hard to watch. Existing tools either watch only one thing, only send an alert, or need an expensive computer or cloud service. A small IoT network has no simple way to notice an attack, react to it and keep reliable proof.

Proposed Solution

ARGUS is one low-cost ESP32 board that works as a guard for the IoT network. In simple words, it does five things:

1. **Watch:** it runs 13 small security checks at the same time.
2. **Decide:** one odd sign gives a WARNING; signs from two different checks within 20 seconds mean a real attack (CRITICAL).
3. **Act:** on a real attack it cuts power to the protected device with a relay, turns the LED red, sounds a buzzer and sends a Telegram message.
4. **Prove:** it writes every event to a log where each line is locked to the previous one, so any change can be found.
5. **Recover and show:** after 30 quiet seconds it re-checks itself and restores the device, and a web dashboard shows everything live.

1.5 Approach

The project was done in five steps, one for each part of the system:

Step	Part of the system	What was done
1	Detection	Wrote 13 independent detection modules for network, protocol, device, data, web and physical threats.
2	Correlation	Built a rule engine that groups events inside a 20-second window and sets the level NOMINAL, WARNING or CRITICAL.
3	Response	Added relay isolation, RGB LED, buzzer, Telegram bot, self-healing and operator modes.
4	Evidence and visibility	Added a SHA-256 hash-chained forensic log and a PIN-protected web dashboard served by the ESP32 itself.
5	Testing	Built a second ESP32 (attacker simulator) with its own dashboard to launch safe test attacks on the lab network.

Table 1.1: Project approach

1.6 Scope of the Project

Within scope: a single defending ESP32 node protecting one load; detection of the 13 attack types listed in Chapter 4; automatic isolation, alerting, logging and recovery; a local dashboard; and a safe attacker simulator for an isolated lab network (phone hotspot or own router).

Outside scope: protecting many nodes at once, encrypted or authenticated broker access, production-grade secure boot, and any testing on networks or devices that the team does not own. The attacker simulator is designed to work only against our own monitor node.

1.7 Organization of the Rest of the Report

- **Chapter 2 – Literature Review** compares earlier approaches and shows the gap that ARGUS fills.
- **Chapter 3 – Hardware Design** presents the architecture of the defending model, components, wiring and the attacker node.
- **Chapter 4 – Software Design** explains the 13 detection modules, correlation, response, forensic log, dashboard and attacker software.
- **Chapter 5 – Results and Discussion** lists the test cases, results and limitations.
- **Chapter 6 – Conclusions and Future Scope** summarises the work and suggests improvements.

Chapter 2
Literature Review

2.1 Previous Approaches to Solve the Problem

Securing IoT networks has been tried in several ways. Each approach is useful, but each leaves a gap that ARGUS tries to close.

Network intrusion detection systems

Tools such as Snort inspect network traffic against known attack signatures [10]. They are powerful, but they usually run on a full computer and need an expert to set up and read the output. They also do not look inside the small device itself (its firmware, memory or case).

Cloud-based IoT security platforms

Cloud platforms collect data from many devices and analyse it remotely. They depend on a working internet link and an account, they add running cost, and they react slowly when the attack is local to the Wi-Fi network.

Single-purpose detectors

Many hobby and student projects use an ESP32 to detect one thing, for example Wi-Fi deauthentication frames or a fake access point. These are cheap and easy, but they watch only one symptom, they do not combine evidence from several sources, and they usually stop at sending an alert.

Device-level protections

Secure boot and flash encryption protect the firmware of a single chip [2]. They stop some attacks from succeeding but do not watch the network, and they do not tell the owner that someone is trying.

Zero-trust thinking

Zero-trust architecture says that no device or message should be trusted just because it is inside the network; every request must be verified [6]. This idea fits IoT well, but it is rarely applied on very small devices.

Approach	Strength	Gap
Network IDS (e.g. Snort) [10]	Rich signature detection	Needs a PC and an expert; blind to the device itself
Cloud IoT platforms	Central view of many devices	Needs internet and account; slow local reaction
Single-purpose ESP32 detectors	Cheap and simple	One symptom only; alert but no action
Secure boot / flash encryption [2]	Protects firmware on one chip	No network view; no alerting
ARGUS (this project)	13 checks, correlation, automatic response, tamper-evident log, local dashboard on one ESP32	Protects one node; see Section 5.3

Table 2.1: Comparison of approaches

ARGUS combines the useful ideas of these approaches on a single low-cost board: many checks across layers (as in an IDS), zero-trust verification of device messages [6], hash-protected logging in the spirit of secure logs [11], and an automatic physical response.

Chapter 3
Hardware Design

3.1 Overview of the Two-Node Setup

The project uses two ESP32 boards on one isolated 2.4 GHz Wi-Fi network (a phone hotspot or personal router). A laptop optionally runs the Mosquitto MQTT broker, and a phone shows the dashboard and receives Telegram alerts.

Node	Board	Role
Defending node (ARGUS Monitor)	ESP32 DevKit V1 with DHT sensor, relay, RGB LED, buzzer and tamper wire	Watches the network and itself, decides, responds, logs and serves the dashboard
Attacker node (ARGUS Attacker)	Second ESP32 DevKit V1, no extra parts	Creates safe, labelled test attacks against the monitor; has its own web dashboard

Table 3.1: The two nodes

3.2 Architecture of the Defending Model

Figure 3.1 shows the architecture of the defending node. Data flows from the inputs through detection, correlation and response, and a fifth layer shows the result to the operator. Table 3.2 summarises each layer.

Layer	What it does
1 Inputs	Raw information the ESP32 can see: Wi-Fi scans and events, its ARP table, MQTT messages, its own firmware image, the DHT sensor, a touch pin, web requests and system state (memory, reset reason).
2 Detection	13 independent modules, grouped as Network, Protocol, Device, Data, Web and Physical. Each module looks for one kind of attack and raises an event.
3 Correlation	Collects events in a 20-second window. One module gives WARNING; two or more different modules give CRITICAL; a firmware change is CRITICAL at once. Also gives a threat score from 0 to 100.
4 Response	On CRITICAL: relay cuts the protected load, RGB LED and buzzer alarm, Telegram alert, entry in the hash-chained log. After 30 quiet seconds the self-heal step re-checks and restores the load.
5 Visibility	A web dashboard served by the ESP32 (argus.local) and Telegram on the operator’s phone.

Table 3.2: Layers of the defending model

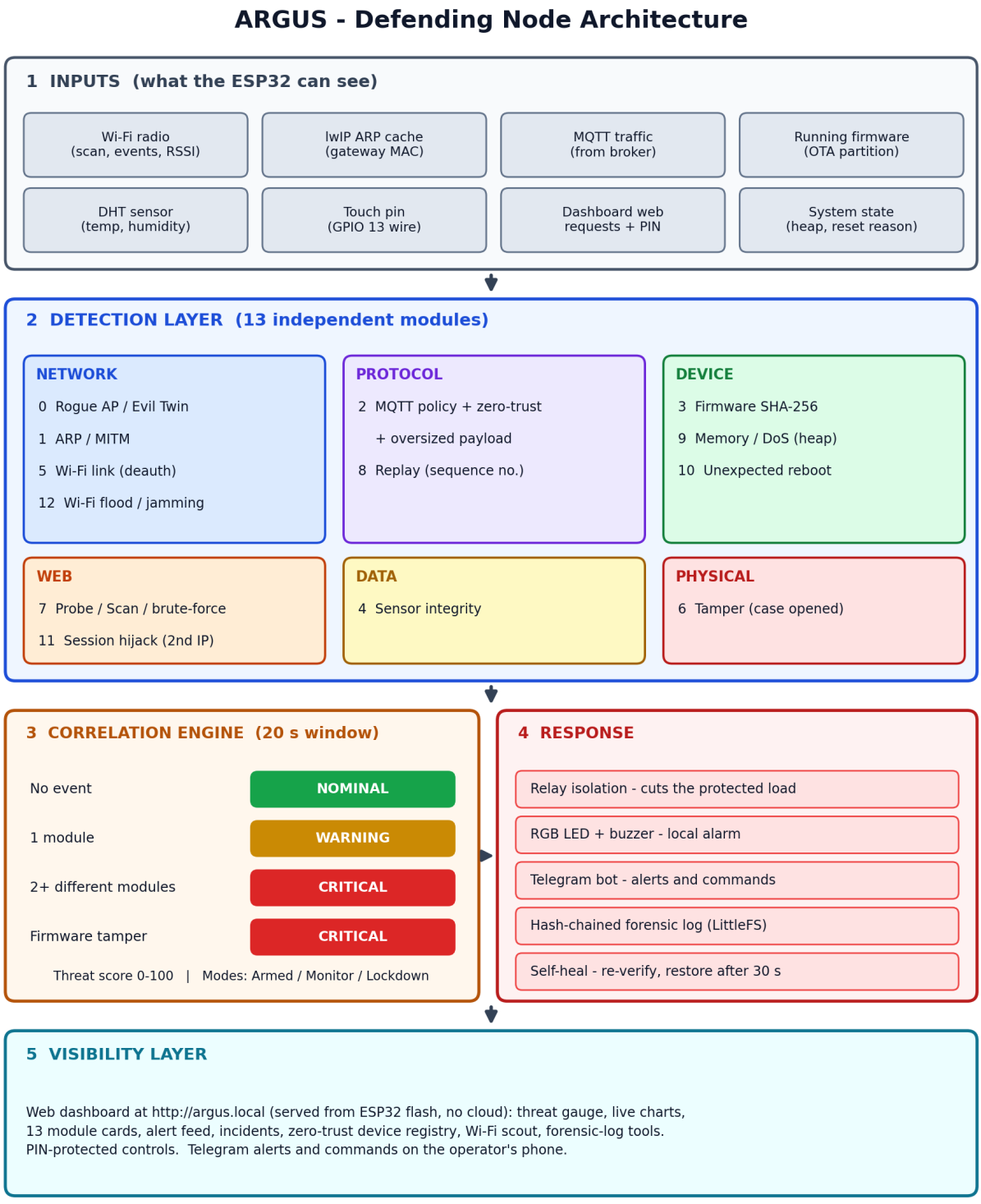


Figure 3.1: Architecture of the ARGUS defending node

3.3 Defending Node: Components and Wiring

Component	Qty	Used for
ESP32 DevKit V1	1	Main controller, Wi-Fi, web server
DHT temperature and humidity sensor (DHT22; DHT11 also supported)	1	Sensor whose data is protected
5 V, 1-channel relay module	1	Isolates the protected load
RGB LED (common anode) and 3 × 220 Ω resistors	1 set	Status light: green, yellow, red
Active buzzer (5 V)	1	Local alarm
Loose jumper wire on a touch pin	1	Tamper (case-opened) sensor
Protected load: 5 V mini fan, or LED + 100 Ω	1	The device that ARGUS protects
Breadboard, jumper wires, USB cable and power source	1 set	Connections and power

Table 3.3: Components of the defending node

The on-board BOOT button is also used: a short press silences the alarm and a 3-second hold releases the isolation. Figure 3.2 shows the complete wiring and Table 3.4 lists the connections.

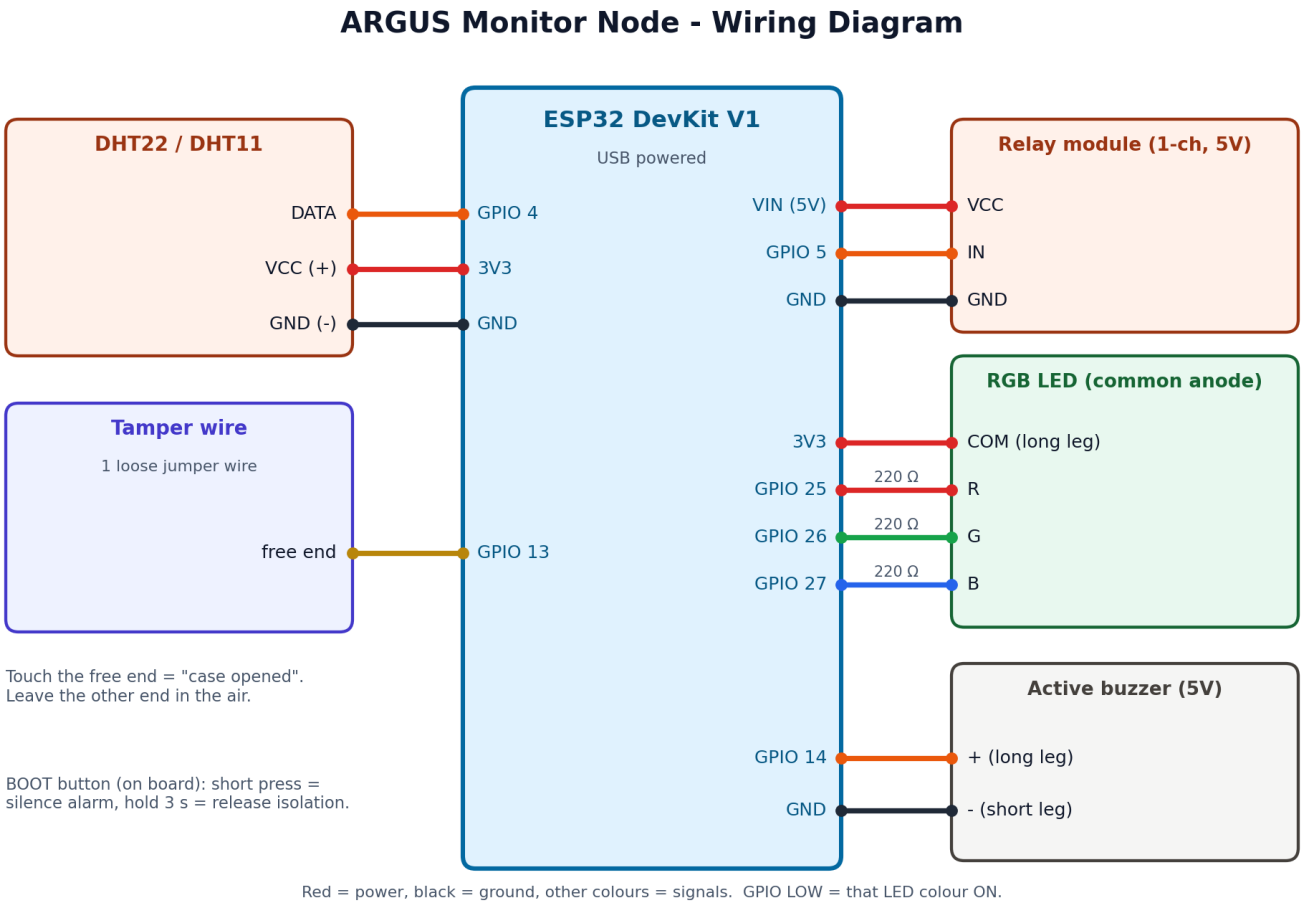


Figure 3.2: Wiring of the defending node

Part	Part pin	ESP32 pin	Note
DHT sensor	DATA / VCC / GND	GPIO 4 / 3V3 / GND	
Relay	IN	GPIO 5	Active-low by default
Relay	VCC / GND	VIN (5 V) / GND	Board powered by USB
RGB LED	R / G / B	GPIO 25 / 26 / 27	Each through 220 Ω
RGB LED	Common (long leg)	3V3	Common anode; set LED_COMMON_ANODE for other types
Buzzer	+ / −	GPIO 14 / GND	
Tamper wire	Free end	GPIO 13	Capacitive touch input
BOOT button	On board	GPIO 0	Silence / release

Table 3.4: Pin connections of the defending node

The protected load is wired through the relay's COM and NC (normally closed) contacts, so the load runs while the relay coil is off and stops when ARGUS energises the coil. This is also a fail-safe for normal operation: if ARGUS is idle, the device simply works.

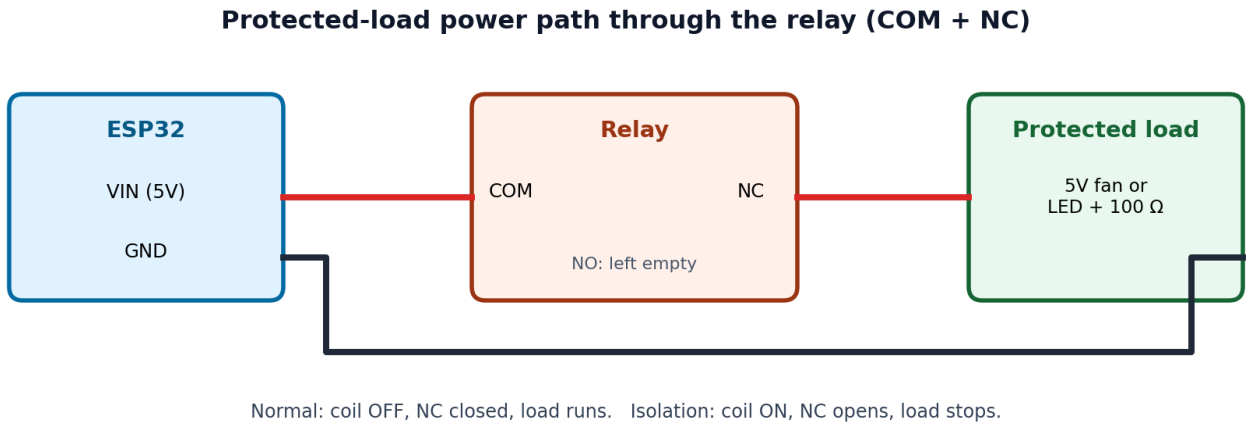


Figure 3.3: Power path of the protected load

Figure 3.4 shows the assembled defending IoT model.

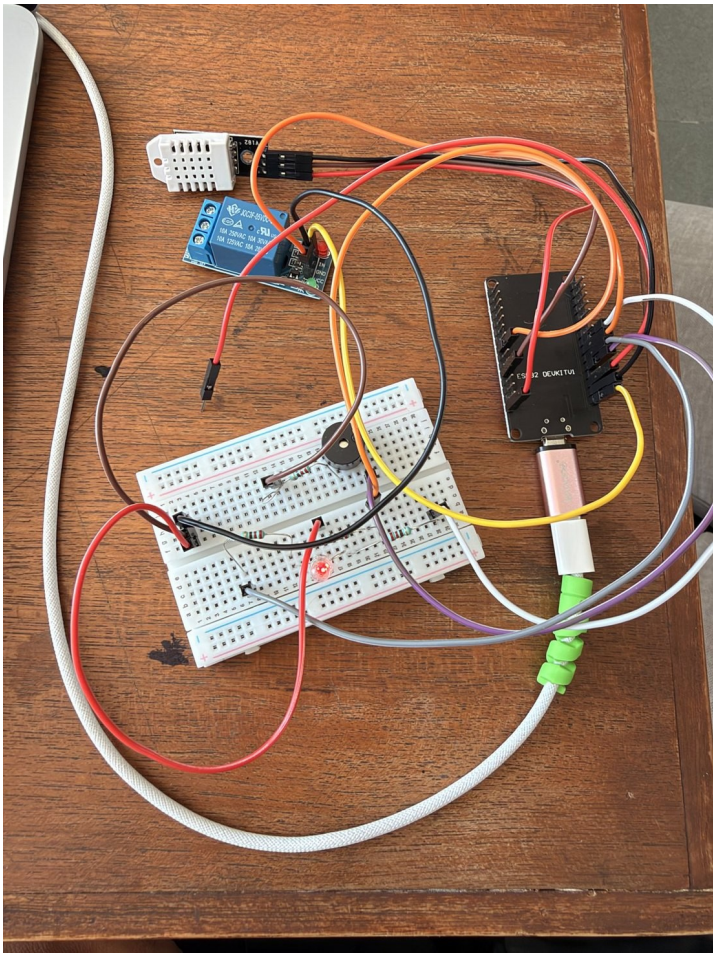


Figure 3.4: Assembled hardware of the defending IoT model

3.4 Attacker Node

The attacker node is a second ESP32 DevKit V1 with no extra hardware. It joins the same hotspot, publishes MQTT messages, can start a fake access point with the same Wi-Fi name, and sends HTTP requests to the monitor's dashboard. It is intended for use only on our own isolated lab network. Figure 3.5 shows the attacker model: a bare ESP32 board powered over USB.

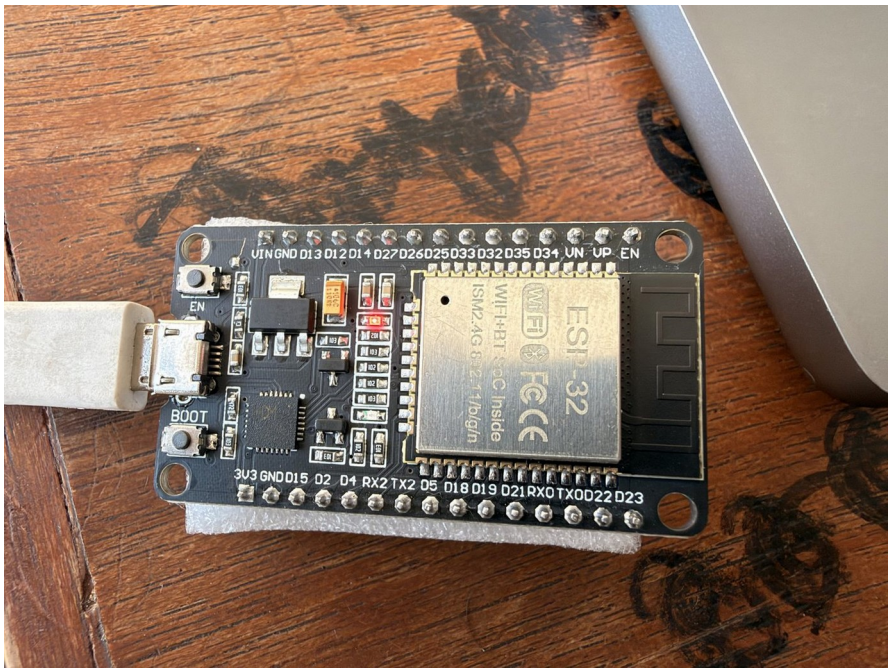


Figure 3.5: Hardware of the attacker IoT model

3.5 Test Network

Element	Purpose
Phone hotspot (2.4 GHz)	Isolated test network shared by all devices. College Wi-Fi is not used because it blocks device-to-device traffic.
Laptop with Mosquitto broker (optional)	MQTT broker on port 1883 for the sensor and attacker messages.
Phone / laptop browser	Opens the dashboards at argus.local and the attacker's own address.
Telegram bot (optional)	Receives alerts and accepts operator commands from one configured chat.

Table 3.5: Test network elements

Chapter 4
Software Design

4.1 Software Environment

- **Firmware:** C++ (Arduino framework) for ESP32 in Arduino IDE; the monitor is split into Argus_Monitor.ino (logic) and dashboard_html.h (the web page stored in flash).
- **Libraries:** DHTesp (sensor), PubSubClient (MQTT), and the ESP32 core libraries WiFi, WebServer, ESPmDNS, HTTPClient, LittleFS, Preferences, mbedTLS (SHA-256), esp_ota_ops and lwIP [2].
- **Services:** Mosquitto MQTT broker [3] and the Telegram Bot API [8] (both optional).
- **Dashboard:** HTML, CSS and JavaScript with canvas charts, served by the ESP32 at argus.local, with a JSON API (Appendix B).

4.2 Detection Layer: The 13 Modules

Each module runs independently and raises an event with a severity. Extra "detail" lines are also logged for threshold-based modules, so the evidence shows every contributing event as well as the final verdict. Detail lines are low severity and can never cause isolation on their own.

#	Module	Category	Detection method	Severity
0	Rogue AP	Network	Same Wi-Fi name but unknown hardware address (BSSID) in a scan every 15 s, i.e. an evil twin.	High
1	ARP / MITM	Network	Gateway MAC address in the ARP cache [4] changes from the learned value (checked every 2 s).	High
5	Wi-Fi link	Network	Bursts of forced disconnects (deauthentication-style) or a sudden large signal drop.	Medium–High
12	Wi-Fi flood	Network	Sudden jump in the total number of visible networks above the learned normal: beacon flood or jamming.	High
2	MQTT policy	Protocol	Unknown topic or device, wrong or missing device key, message flood (15 msg/s), oversized payload near the 256-byte buffer (overflow probe).	Medium–High
8	Replay	Protocol	Valid key but sequence number not increasing: a captured message sent again.	High
3	Firmware	Device	SHA-256 of the running firmware image [5] differs from the trusted value stored at first boot.	Critical
9	Memory / DoS	Device	Free heap collapses from its high-water mark (or falls below 15 kB): possible memory exhaustion.	High
10	Reboot	Device	Hardware reset reason at boot is panic, watchdog or brown-out instead of normal power-on.	High

#	Module	Category	Detection method	Severity
4	Sensor	Data	Value out of range, jump over 5 °C (or 25 % humidity) in 5 s, sensor disconnected, fake value or foreign publisher on the sensor topic.	Medium–High
7	Probe / Scan	Web	Six or more unknown dashboard URLs in 8 s, or five wrong PINs in a row.	High
11	Session	Web	Dashboard commands from a second IP address within 2 minutes of the first: possible stolen PIN.	High
6	Tamper	Physical	Capacitive touch reading on GPIO 13 deviates from the value learned at boot: case opened.	High

Table 4.1: The 13 detection modules

4.3 Zero-Trust and Replay Protection

Every MQTT device publishes under argus/dev/<id>/… and must prove who it is with a pre-shared key, following the zero-trust idea that nothing inside the network is trusted by default [6]. The payload formats are:

Payload format	Meaning
key|value	Basic zero-trust message: the key identifies the device.
key|seq|value	Signed message with an ever-increasing sequence number. If the key is valid but seq is not larger than the last accepted one, it is flagged as a replayed (captured) packet and not as a fresh reading.

Table 4.2: MQTT payload formats

A wrong, missing or unknown key is a zero-trust violation with higher severity than a plain unknown topic. The dashboard’s device registry lists every device id seen, its last-seen time, trusted or untrusted status, message count and last accepted sequence number, so replay protection is visible live.

4.4 Correlation Engine

The correlation engine is what turns many small signals into one clear decision.

Condition (20 s window)	Result
No events	NOMINAL (green)
Events from exactly one module	WARNING (yellow, beep, Telegram). Never isolates.
Events from two or more different modules	CRITICAL (red): isolate, alarm, log, Telegram
Firmware tamper event	CRITICAL immediately
Currently isolated	Stays CRITICAL until self-heal or operator release

Table 4.3: Correlation rules

A **threat score** from 0 to 100 summarises the current severity, the number of modules involved and how recent the events are. Three **operating modes** are available: **Armed** (auto-isolate), **Monitor only** (alert but never isolate) and **Lockdown** (isolate and hold until the operator releases).

4.5 Response Layer

- **Relay isolation:** the load is on COM + NC, so energising the coil cuts it; polarity is configurable.
- **Local signals:** RGB LED (green, yellow, red patterns) and an active buzzer with mute.
- **Telegram bot:** sends alerts and accepts /status, /isolate, /release, /mute, /unmute, /armed, /monitor and /help only from the configured chat; old updates are ignored at start-up.
- **Self-healing:** after 30 s of quiet, the firmware hash, gateway MAC and Wi-Fi link are re-verified and the load is restored.
- **Physical control:** BOOT button short press silences, 3-second hold releases.

4.6 Tamper-Evident Forensic Log

Each event is stored on the internal LittleFS flash as one line:

seq;time;text|previousHash|hash

where hash = SHA-256(previousHash + entry) [5]. Every line is therefore locked to the one before it, in the manner of cryptographically protected logs [11]. The latest hash and sequence number are also kept in non-volatile memory. The Verify function recomputes the whole chain and reports the first broken line, a sequence gap or a truncation. The file rotates at 40 KB while keeping an anchor so that verification keeps working. Logs created by the dashboard's demo buttons are tagged [SIM].

4.7 Dashboard

The ESP32 serves a single-page dashboard at http://argus.local that works without internet and adapts to a phone screen. It was designed to be attractive and easy to understand:

- **How Argus Decides strip:** four plain-language cards at the top explain WARNING versus CRITICAL, zero-trust and self-heal.
- **Module cards:** each of the 13 modules has an icon and a category tag (Network, Protocol, Device, Data, Web, Physical) so it can be scanned at a glance.
- **Status and charts:** threat gauge, live temperature, humidity and Wi-Fi signal charts, module timeline, filterable alert feed and incident history.
- **Zero-trust device registry** with last sequence number, a **Wi-Fi scout** table of nearby networks, and forensic-log tools (verify, refresh, download).
- **Controls:** PIN-protected (PIN sent in a request header); five wrong attempts lock the controls for 60 seconds. A hardware self-test and simulated-event buttons support demonstration.

Figures 4.1 to 4.5 show the monitor dashboard during testing: first in the NOMINAL state, then during a coordinated attack, followed by the registry, charts, alert feed, incidents and control panel.

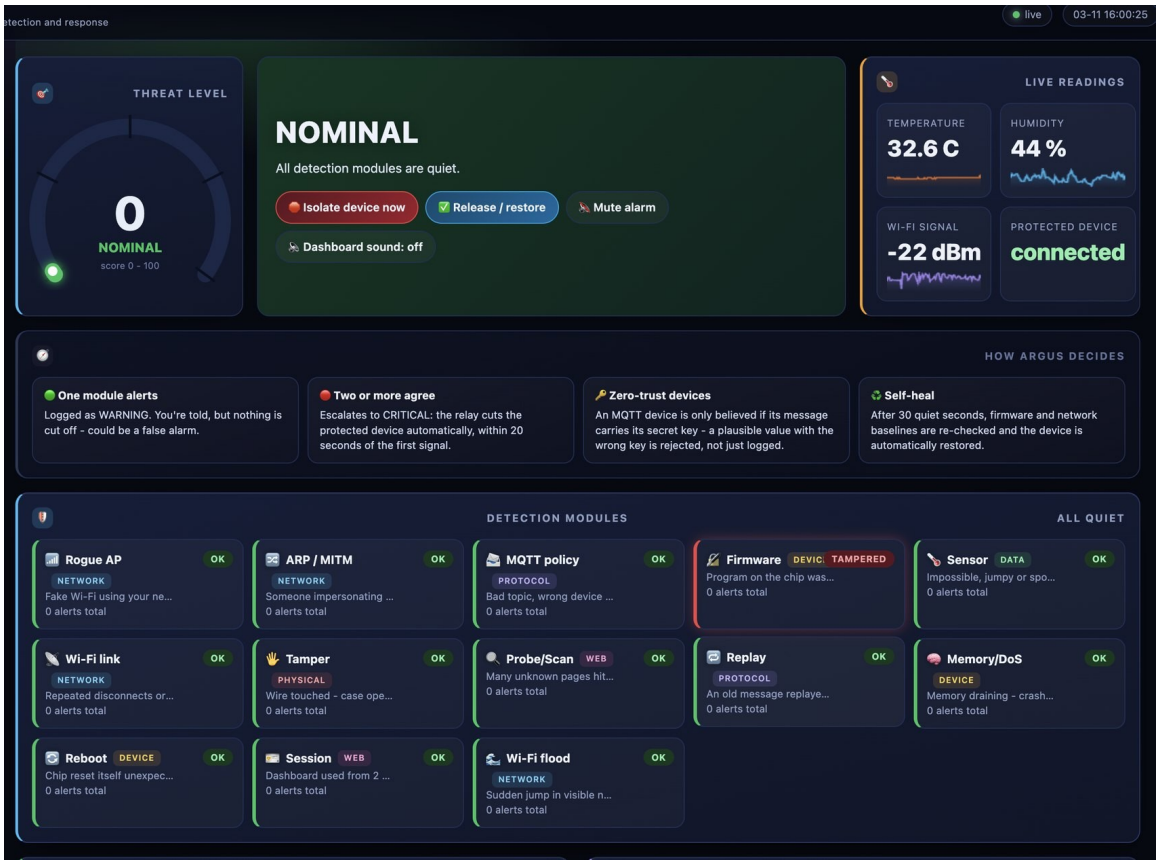


Figure 4.1: Monitor dashboard in the NOMINAL state (score 0, all modules quiet, protected device connected)

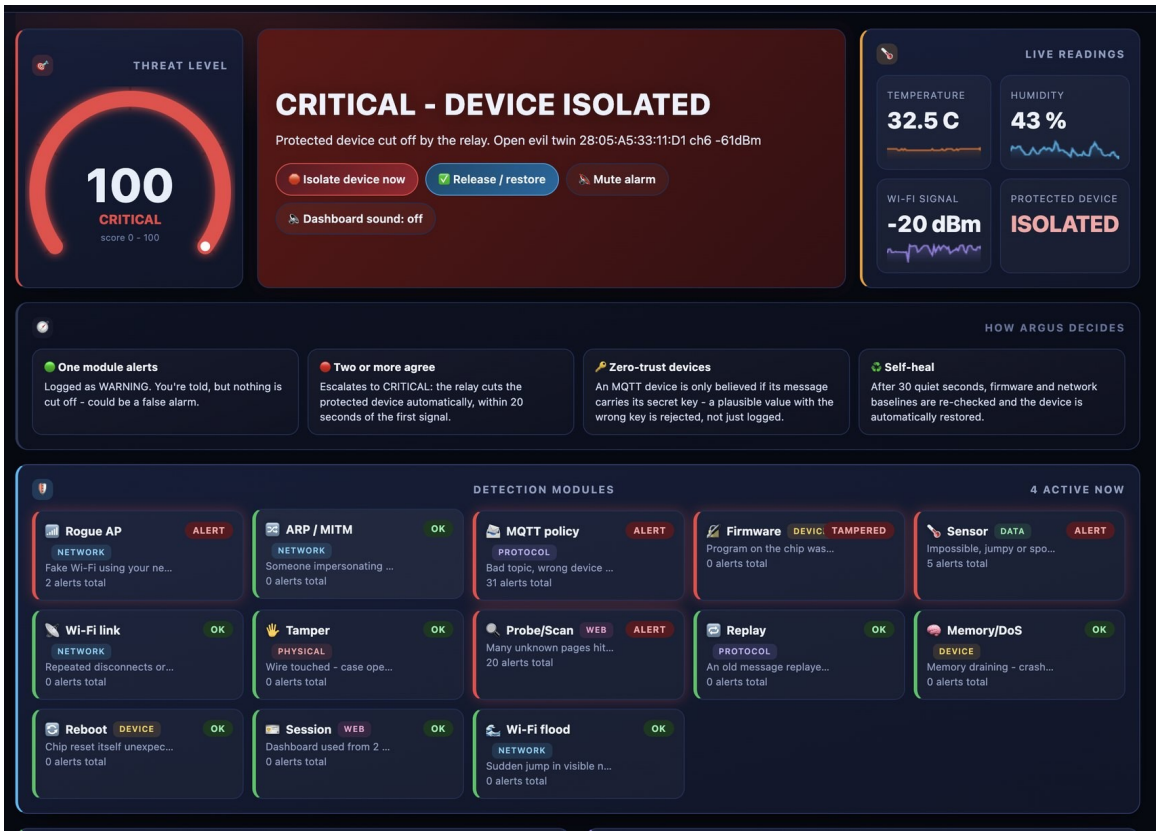


Figure 4.2: Monitor dashboard in the CRITICAL state (score 100, protected device ISOLATED after an open evil twin and MQTT abuse)



Figure 4.3: Zero-trust device registry, Wi-Fi scout and live charts

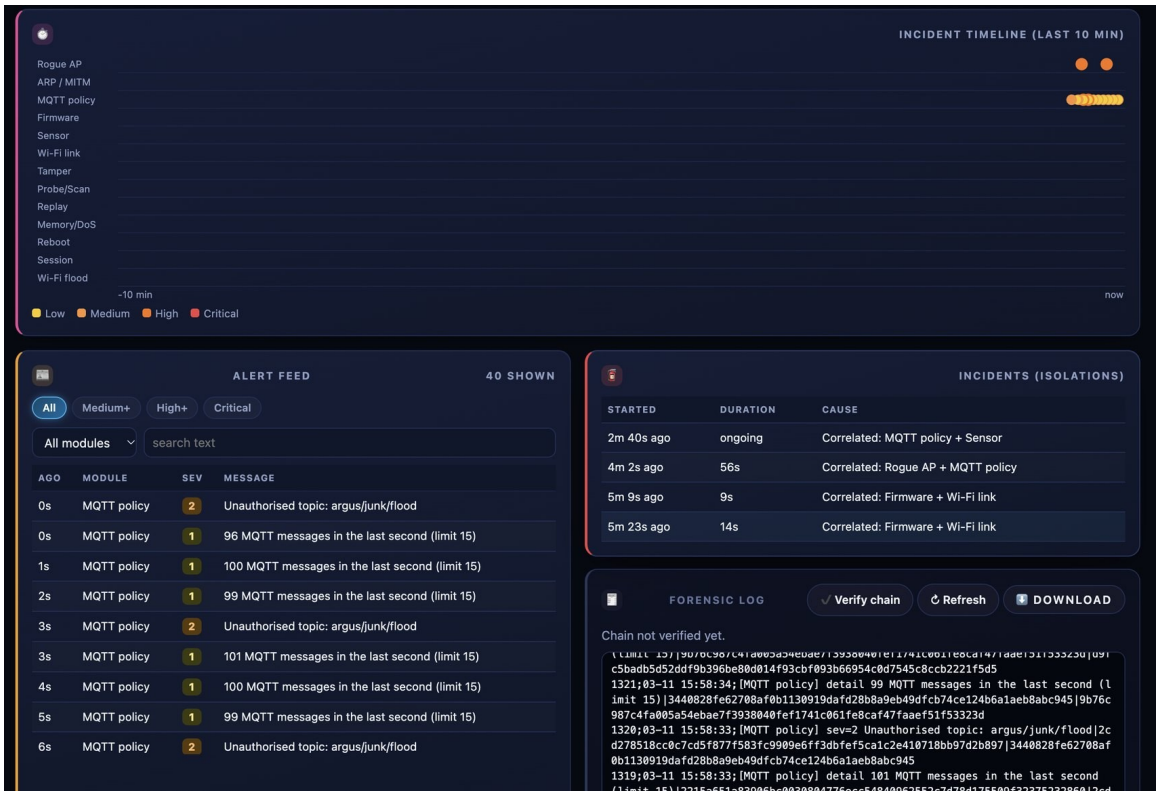


Figure 4.4: Incident timeline, alert feed, incident history and forensic log

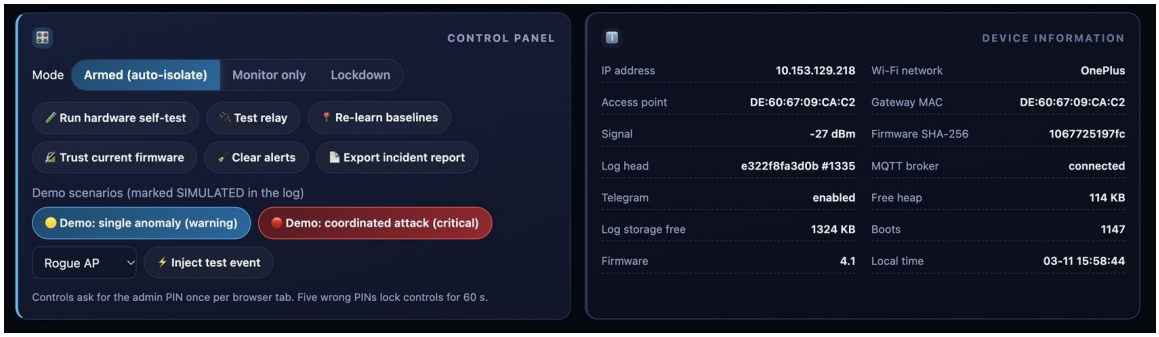


Figure 4.5: Control panel (modes, self-test, demo scenarios) and device information

4.8 Attacker Simulator Software

The attacker firmware serves its own web dashboard (and also accepts Serial Monitor commands), so the whole demonstration can be run from a phone. Each test targets one of the monitor's modules:

Test	Action	Expected module
1	Evil twin: fake access point with the same SSID and a different BSSID	Rogue AP
2	MQTT flood: about 100 messages per second to a junk topic	MQTT policy
3	Unauthorised topic (argus/admin/relay)	MQTT policy
4	Wrong key: spoof sensor1 with a bad device key	MQTT policy
5	Fake value: correct key, implausible 55 °C reading	Sensor
6	Unknown device: publish as sensor9	MQTT policy
7	Probe / scan: six quick requests to unknown dashboard URLs	Probe / Scan
8	Enumerate: publish as sensor10 to sensor14 in a row	MQTT policy
9	PIN brute-force: five wrong PIN attempts on the dashboard API	Probe / Scan

Table 4.4: Attacker simulator tests

ARP spoofing and Wi-Fi deauthentication are real attacks on the LAN and the radio; they are generated from a laptop with standard tools (for example arpspoof and aireplay-ng) on the same isolated network. Figure 4.6 shows the attacker dashboard.

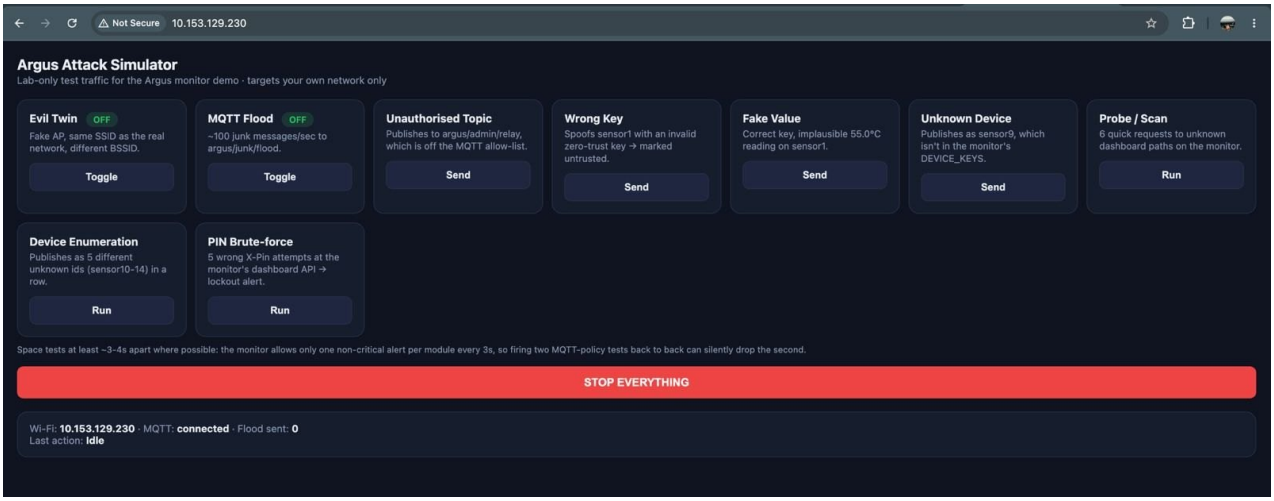


Figure 4.6: ARGUS attacker dashboard (nine test buttons and a STOP EVERYTHING button)

Chapter 5
Results and Discussion

5.1 Test Setup

All tests were planned on the isolated hotspot network with the defending node, the attacker node, a laptop broker and a phone. The monitor dashboard was also tested in a browser with simulated data for all modules; it loaded without JavaScript errors and every module card, the device registry and the “How Argus Decides” strip displayed correctly.

5.2 Test Cases and Results

Table 5.1 lists the test cases. The expected behaviour follows from the design in Chapter 4; the last column records what was observed on the dashboard during testing; the remaining cells are completed during the practical demonstration.

#	Test	Expected result	Observed
1	Normal boot	Green LED, NOMINAL, dashboard loads, self-test passes	
2	Touch the tamper wire	Tamper alert, WARNING only, no isolation	
3	Single attack (e.g. attacker test 5, fake value)	One module: WARNING, no isolation	
4	Two different attacks within 20 s (e.g. evil twin + MQTT flood)	CRITICAL: relay isolates load, alarm, Telegram, incident logged	CRITICAL, device ISOLATED, incident “Rogue AP + MQTT policy” logged (Figs 4.2, 4.4)
5	Stay quiet for 30 s after an incident	Self-heal restores the load; Telegram notice	Incidents ended; back to NOMINAL, device connected (Fig 4.1)
6	Evil twin (attacker test 1)	Rogue AP alert	Open evil twin detected on channel 6 (Fig 4.2)
7	Wrong key and unknown device (tests 4 and 6)	MQTT policy alerts (zero-trust violation, unknown device)	sensor9–sensor13 shown UNTRUSTED; sensor1 trusted (Fig 4.3)
8	Replay a signed message with an old sequence number	Replay alert	
9	Unplug the sensor data wire	Sensor alert (not responding)	
10	Probe / scan and PIN brute-force (tests 7 and 9)	Probe/Scan alerts; controls locked for 60 s after 5 wrong PINs	
11	Dashboard commands from a second IP within 2 min	Session alert	

#	Test	Expected result	Observed
12	Reflash modified firmware	Firmware tamper at boot, immediate CRITICAL	
13	Verify the log (untouched), then edit one line and verify again	OK first; “BROKEN at line n” after edit	
14	Telegram /isolate and /release	Relay switches; replies received	

Table 5.1: Test cases

The screenshots in Chapter 4 give direct evidence of the correlation behaviour. In Figure 4.2 an open evil twin (a fake network with the same name, on channel 6) and an MQTT flood of about 100 messages per second were seen by two different modules, so the engine raised CRITICAL with a threat score of 100 and the relay isolated the protected device. The incident history in Figure 4.4 lists the cause of each isolation as a correlation of two modules, for example “Rogue AP + MQTT policy”. The MQTT flood also produced one low-severity detail line per second, so the evidence shows the attack rate and not only the final verdict. After the attack stopped, the system returned to NOMINAL and the device was reconnected (Figure 4.1).

5.3 Discussion and Limitations

The tests show the main idea of the design: a single abnormal event only warns, which avoids cutting power for a false alarm, while two different modules agreeing inside the 20-second window is strong evidence of a real attack and triggers isolation. Detail logging makes it easy to see exactly why a verdict was given. The following limitations should be kept in mind:

- The firmware was compile-checked and the dashboard tested with simulated data; hardware behaviour such as relay polarity must be confirmed on the actual board.
- The firmware hash and log head are stored on the same device, so an attacker with physical access could erase them. The Telegram boot message anchors them off-device. A production system needs secure boot and flash encryption [2].
- The ARP check protects the monitor itself, not other hosts on the network.
- MQTT checks identify abnormal topics, keys and rates, not client identities; broker access control lists and TLS would complement them.
- The Telegram link does not pin certificates (a demo simplification), and commands from a group chat can be issued by any member.
- Several MQTT alerts share one module with a 3-second alert rate limit, so two quick attacks of the same kind may appear as one alert.

Chapter 6
Conclusions and Future Scope

6.1 Conclusions

ARGUS shows that useful, multi-layer IoT security does not need expensive equipment. With one ESP32 and a few low-cost parts it watches the network, the protocol, the device, the data, the web interface and the hardware case through 13 detection modules. It decides by correlation rather than by single alarms, responds on its own by isolating the protected device and alerting the operator, writes a tamper-evident forensic log, and recovers by itself when the danger has passed. A clear local dashboard makes all of this easy to understand, and the attacker simulator allows the complete behaviour to be demonstrated safely.

The project combines network security, embedded systems, digital forensics and security automation in one small, low-cost system.

6.2 Future Scope

- Authenticated and encrypted MQTT (broker login, access control lists and TLS).
- Secure boot and flash encryption, with the log head anchored off-device.
- Source tracking of attacks and learned (adaptive) baselines instead of fixed thresholds.
- Several protected nodes reporting to one ARGUS dashboard.
- Protected management frames (IEEE 802.11w) support and an ARP check that also protects other hosts [7].

References

[1] M. Antonakakis et al., “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, Vancouver, BC, Canada, 2017, pp. 1093–1110.

[2] *ESP32 Series Datasheet*, Espressif Systems, Shanghai, China, 2023.

[3] A. Banks and R. Gupta, Eds., *MQTT Version 3.1.1*, OASIS Standard, Oct. 2014.

[4] D. C. Plummer, “An Ethernet Address Resolution Protocol,” IETF, RFC 826, Nov. 1982.

[5] *Secure Hash Standard (SHS)*, FIPS PUB 180-4, National Institute of Standards and Technology, Aug. 2015.

[6] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST SP 800-207, Aug. 2020.

[7] *IEEE Standard for Information Technology – Local and Metropolitan Area Networks – Amendment 4: Protected Management Frames*, IEEE Standard 802.11w-2009, 2009.

[8] *Telegram Bot API*, Telegram Messenger. [Online]. Available: https://core.telegram.org/bots/api

[9] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, “DDoS in the IoT: Mirai and other botnets,” *Computer*, vol. 50, no. 7, pp. 80–84, Jul. 2017.

[10] M. Roesch, “Snort: Lightweight intrusion detection for networks,” in *Proc. 13th USENIX Systems Administration Conf. (LISA)*, Seattle, WA, USA, 1999, pp. 229–238.

[11] B. Schneier and J. Kelsey, “Cryptographic support for secure logs on untrusted machines,” in *Proc. 7th USENIX Security Symp.*, San Antonio, TX, USA, 1998, pp. 53–62.

Appendix

A. Key tuning parameters of the defending node

Parameter	Value	Meaning
Correlation window	20 s	Events this close together are treated as related
Quiet time for self-heal	30 s	Calm time needed before the load is restored
Per-module alert rate limit	3 s	Anti-spam debounce per module
Rogue AP scan interval	15 s	Asynchronous Wi-Fi scan
ARP check interval	2 s	Gateway MAC comparison
Heap check interval	4 s	Memory / DoS watch
MQTT flood threshold	15 msg/s	Treated as a flood
Wi-Fi flood rule	≥ 12 networks and ≥ 6 above normal	Beacon flood / jamming signature
Wrong-PIN limit	5 attempts, 60 s lock	Brute-force protection
Log rotation size	40 KB	Forensic log file size limit

Table A.1: Tuning parameters

B. Dashboard JSON API of the defending node

Endpoint	Purpose
/api/status	Current level, score, module states, device information
/api/history	Data for the live charts
/api/log, /api/log.txt	Latest forensic log entries / full log download
/api/verify	Recomputes and checks the hash chain
/api/devices	Zero-trust device registry
/api/scan	Wi-Fi scout results
/api/cmd	PIN-protected control commands

Table B.1: Dashboard API

C. Serial Monitor commands of the defending node

help, status, verify, sim <0–12>, isolate, release, trust, rebase, wipe (115200 baud).

D. Notes on use

Credentials, tokens and keys (Wi-Fi password, admin PIN, Telegram bot token and chat id, device keys) are set in the settings block at the top of the firmware and are intentionally not reproduced in this report. The attacker simulator must be used only on a network and devices owned by the team.